

KUNROUTE

LIGHTWEIGHT LLM ROUTING ON KUNPENG CPU

玄枢

智能体网管系统技术方案

历史数据学习 · 质量优先 · 成本最小 · 延迟优化

先筛出质量达标模型，
再选择成本最低者。

成本等价时选择延迟最低模型；一次并行评估 5~10 个候选。

五元组历史数据

$QUALITY \geq \tau$

NEON / MULTICORE



质量先过线



成本再最小



同成本选最快

竞赛提交版 · 算法设计 · 鲲鹏实现 · 完整工程包

$D = (\text{query}, \text{model}, \text{quality_score}, \text{latency}, \text{cost})$

Model Pool · $M = 5 \sim 10$ | Platform · Kunpeng CPU / AArch64

Document · Final Submission | 2026-09

玄

KUN

1. 方案总览：严格对应命题的四步闭环

KunRoute 面向 5~10 个异构大模型，利用历史五元组 D 学习 query 与候选模型之间的质量、成本和延迟关系。新请求到达后，Router 一次提取请求特征，一次批量评估全部候选模型，并严格按“质量达标 -> 成本最低 -> 延迟最低”的顺序输出 model*。在线推理部署在鲲鹏 CPU，核心矩阵乘加使用 NEON，多请求使用多核并行。

一句话方案 历史数据训练 Q/C/L 预测器；质量先过线，成本再最小，成本等价时选择延迟最低模型。

命题要求	KunRoute 实现
M=5~10 个模型	标准配置提供 Model1~Model10；模型表包含单位成本、平均延迟和任务标签
历史数据 D	训练输入严格采用 (query, model, quality_score, latency, cost)
新请求输出 model*	并行预测所有候选模型 Q/C/L 后执行三级字典序决策
利用历史信息	从 D 聚合 model×task 质量先验，并训练请求级 Q/C/L 条件预测
在线性能估算	近期质量 EWMA 修正 Q，近期延迟 EWMA 修正 L
鲲鹏 CPU	AArch64 LightMLP；NEON 乘加；OpenMP 请求级多核并行
答题材料	特征设计表、路由器架构图、核心训练与在线路由伪代码完整给出

2. 数学建模：质量是门槛，成本与延迟按顺序优化

设模型集合 $M=\{1,...,M\}$ ， $M\in[5,10]$ 。对新请求 $x=query_new$ ，用户质量阈值为 τ 。Router 对每个模型 m 预测响应质量、调用成本和端到端延迟。

$$Q_hat_m(x), C_hat_m(x), L_hat_m(x), m = 1...M$$

验证集上统计质量过预测残差，得到模型×任务校准量 Delta。在线使用校准后的 Q_safe 进入质量判断。

$$Q_safe_m(x) = clip(Q_hat_m(x) - Delta_m(task), 0, 1)$$

$$F(x) = \{ m \mid Q_safe_m(x) \geq \tau \}$$

$F(x)$ 是质量可行集合。

$$model* = lexargmin_{(m \in F(x))} (C_hat_m(x), L_hat_m(x))$$

先比较成本；成本进入 1% 等价区间后比较延迟。

当 $F(x)$ 为空时，选择 Q_safe 最高模型，使系统始终输出一个目标模型。该规则保持“质量优先”的排序原则。

$$F(x)=empty \Rightarrow model* = argmax_m Q_safe_m(x)$$

3. 输入数据与模型池

历史主表严格使用命题五元组。工程包内置 500 个 query × 10 个模型的标准数据集，共 5,000 条记录，用于完整跑通训练、评估和路由流程。

字段	含义	模型中的作用
query	请求文本	提取结构、任务和哈希语义特征
model	被调用模型	连接对应模型属性
quality_score	0~1 质量分	训练质量头
latency	端到端延迟 ms	训练延迟头
cost	单次调用成本	训练成本头

模型	单位成本（标准化）	平均延迟 ms	擅长任务标签
Model1	1.00	680	抽取 / 摘要
Model2	1.20	740	翻译 / 问答
Model3	1.55	820	代码 / 抽取
Model4	1.90	900	数学/推理 / 问答
Model5	2.30	960	问答 / 摘要
Model6	2.90	1030	代码 / 问答
Model7	3.70	1140	数学/推理 / 问答
Model8	4.50	1250	问答 / 摘要
Model9	5.80	1380	问答 / 摘要
Model10	7.20	1540	问答 / 数学/推理

模型属性直接参与路由 单位成本和平均延迟是命题已知条件；任务标签用于计算 query×model 专长匹配特征。

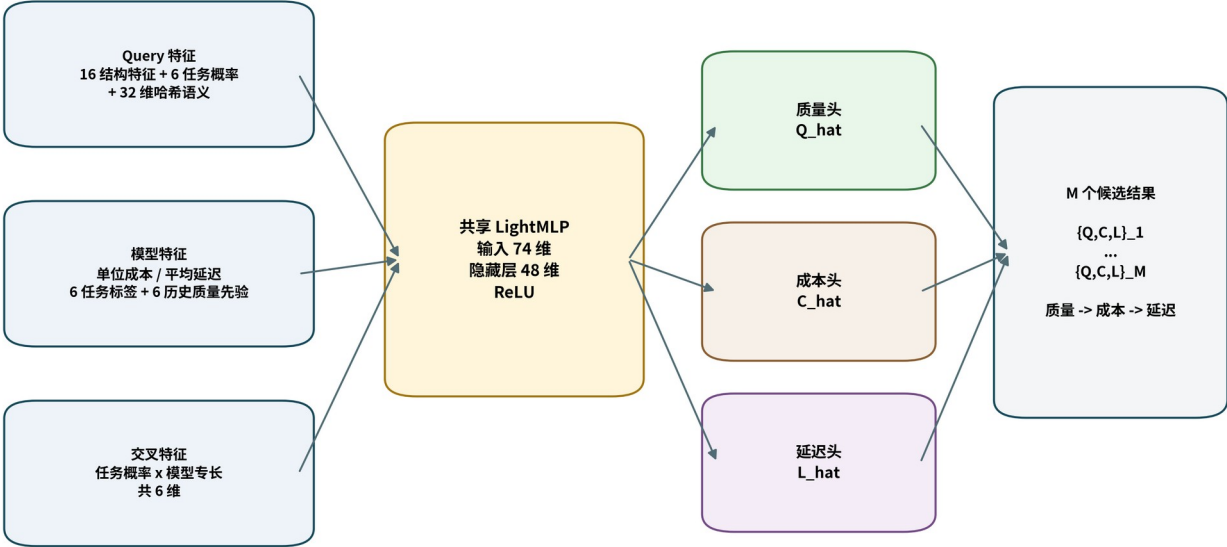
4. 特征设计表

特征设计只围绕“请求是什么、模型擅长什么、历史上表现如何、当前运行状态如何”四个问题展开。LightMLP 的最终输入维度为 74。

特征组	具体特征	维度	用途
请求结构	字节长度、数字/字母/空格/换行/标点比例、代码符号、数学符号、JSON/URL、行数、近似 token 数	16	描述长度和文本形态
任务概率	问答、数学/推理、代码、摘要、翻译、抽取	6	匹配任务类型
哈希语义	UTF-8 字节 bigram hashing + L2 归一化	32	低成本捕获词形与局部语义模式
模型静态属性	单位成本、平均延迟、6 维任务专长标签	8	直接使用命题已知条件
历史质量先验	由 D 聚合 model×task 平均 quality_score	6	利用历史请求结果
任务-模型交叉	任务概率 × 模型专长逐维乘积	6	刻画 query 与模型能力匹配

74 = 16 + 6 + 32 + 8 + 6 + 6

5. 轻量路由器架构



单个 Router 学习 query x model 的条件性能，而不是直接记忆 model_id 标签

图 1 共享模型条件 LightMLP 架构

Query 特征只计算一次。每个候选模型拼接自身属性和历史质量先验，进入同一个 LightMLP。M=5~10 时只增加候选行数，不增加 Router 数量。

$$z_m = \text{concat}(\text{query_feature}, \text{model_feature_m}, \text{task_prob} \times \text{skill_m})$$

$$(q_m, c_m, l_m) = \text{LightMLP}(z_m)$$

层	尺寸	激活/输出
输入层	74	标准化特征
隐藏层	48	ReLU
质量头	1	clip 到 [0,1]
成本头	1	log1p 训练 / expm1 还原
延迟头	1	log1p 训练 / expm1 还原

6. 历史数据训练与质量校准

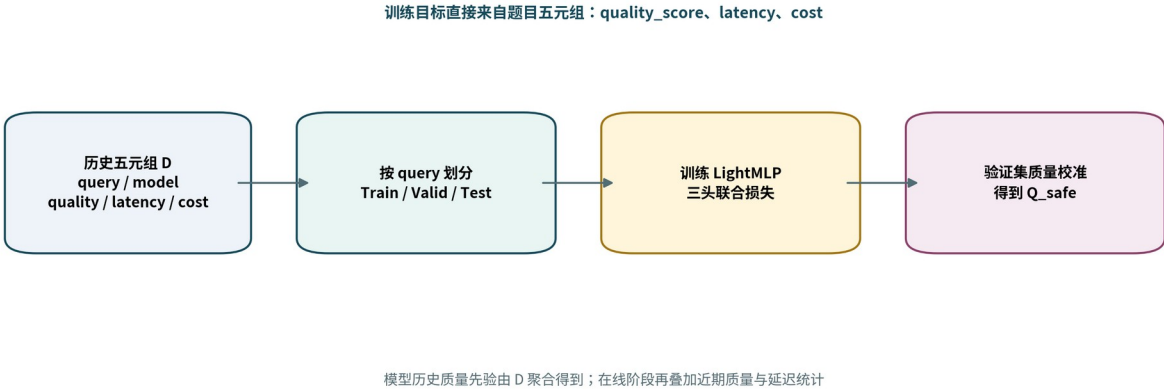


图 2 从历史五元组到 LightMLP 的训练流程

训练按 query 划分 Train / Valid / Test。若同一 query 在多个模型上有记录，这些记录保持在同一分区。三头联合学习质量、成本、延迟，质量头赋予更高损失权重。

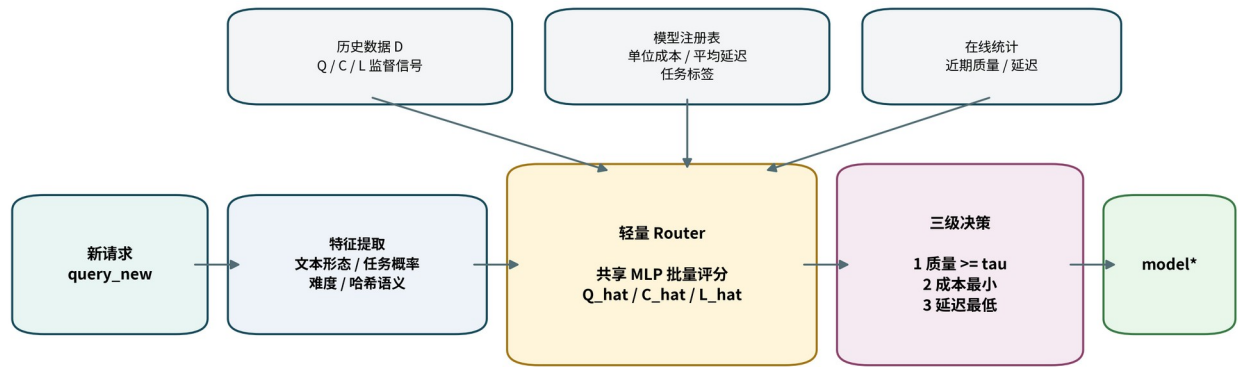
$$\text{Loss} = 4 * L_{\text{quality}} + 1 * L_{\text{cost}} + 0.7 * L_{\text{latency}}$$

目标	训练标签	损失
质量	quality_score	MSE/Huber；验证集计算过预测残差分位数
成本	log1p(cost)	MSE/Huber
延迟	log1p(latency/1000)	MSE/Huber

核心训练伪代码 算法 1 TRAIN_ROUTER(D, ModelTable, tau)

- 1 按 query 划分 Train / Valid / Test
- 2 从 D 聚合 model x task 历史质量先验
- 3 对每条 (query, model, quality, latency, cost)：
- 4 x <- QUERY_FEATURES(query)
- 5 m <- MODEL_FEATURES(model)
- 6 z <- CONCAT(x, m, TASK_PROB(x) x SKILL(m))
- 7 q_hat, c_hat, l_hat <- LightMLP(z)
- 8 更新 Loss = 4*Lq + Lc + 0.7*Ll
- 9 在 Valid 上计算 Delta_(model,task)
- 10 保存 Router 权重、标准化参数、质量校准表

7. 在线路由算法



在线热路径：query 只编码一次，5~10 个模型作为候选批量评分

图 3 新请求的端到端路由流程

新请求进入后，Router 批量得到 5~10 个候选的 Q/C/L。近期质量和延迟用 EWMA 与离线预测融合，形成在线模型性能估算。

$$Q_{\text{online}} = (1 - \rho_q) \cdot Q_{\text{hat}} + \rho_q \cdot \text{EWMA_quality}$$

$$L_{\text{online}} = (1 - \rho_l) \cdot L_{\text{hat}} + \rho_l \cdot \text{EWMA_latency}$$

核心在线伪代码 算法 2 ROUTE(query_new, tau)

```

1 qfeat <- EXTRACT_FEATURES(query_new)
2 对 m = 1...M 批量计算 q_hat[m], c_hat[m], l_hat[m]
3 q_online[m] <- 融合近期质量 EWMA
4 l_online[m] <- 融合近期延迟 EWMA
5 q_safe[m] <- q_online[m] - Delta[m, task]
6 F <- {m | q_safe[m] >= tau}
7 若 F 为空：返回 argmax q_safe
8 c_min <- min(c_hat[m]), m in F
9 F_cost <- {m in F | c_hat[m] <= c_min * (1 + 0.01)}
10 model* <- argmin l_online[m], m in F_cost
11 返回 model*
  
```

决策阶段复杂度为 $O(M)$ 。M 最大为 10，因此三级筛选开销远小于一次大模型调用。

8. 鲲鹏 CPU 推理加速

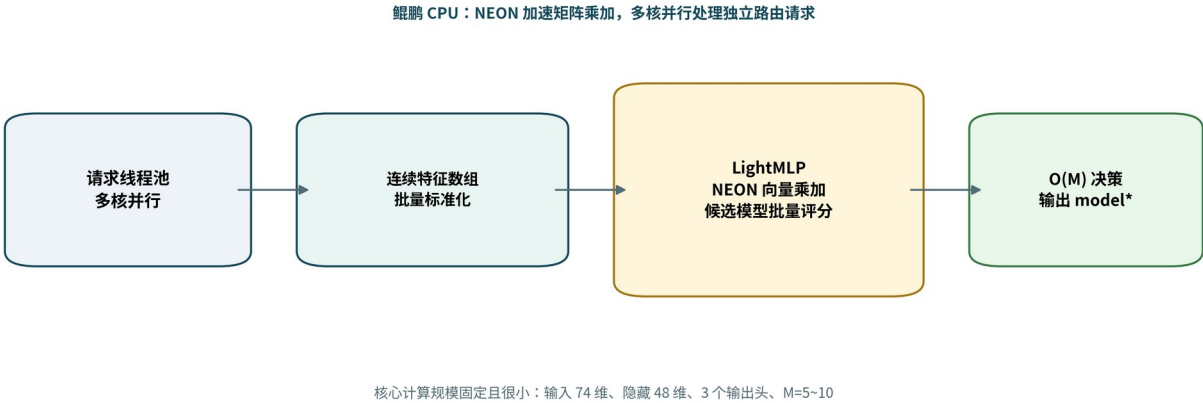


图 4 鲲鹏 CPU 在线推理路径

优化点	实现方式	对应命题要求
NEON 乘加	AArch64 `arm_neon.h`；vld1q_f32 / vmlaq_f32 / vaddvq_f32	利用 NEON 加速 LightMLP 点积与矩阵乘加
候选批量评分	Model1~ModelM 特征连续存储；同一 query 一次计算全部候选	降低 5~10 模型评分开销
多核并行	OpenMP 请求级 parallel for；不同 CPU 核处理独立 query	利用鲲鹏多核提升吞吐
轻量内存布局	连续 float32 数组；权重只读复用；固定输入/隐藏维度	减少分配与缓存开销

```
编译与运行 cmake -S cpp -B build -DCMAKE_BUILD_TYPE=Release；AArch64 自动启用 -march=armv8-a+simd；通过 OMP_NUM_THREADS=1/2/4/8/16 调整并行核数。
```

9. 评测体系

评测顺序与优化优先级一致：先看质量约束，再看成本，最后看延迟。工程包提供 evaluate.py，对冻结测试 query 逐条比较 Router 选择与同一 query 的全模型记录。

指标	定义	用途
Quality Pass Rate	selected quality_score >= tau 的请求占比	检验最高优先级质量约束
Quality Violation Rate	1 - Quality Pass Rate	观察质量越线
Average Cost	所选模型 cost 的均值	比较降本能力
p95 Latency	所选模型 latency 的 95 分位	比较尾部延迟
Oracle Route Accuracy	Router 选择 = 质量合格模型中最低成本、同成本最低延迟模型	直接衡量路由正确性
Cost Regret	Router cost - Oracle cost	衡量与理论最优路由的成本差

基线	规则
All-Strong	固定选择历史平均质量最高模型

基线	规则
All-Cheap	固定选择单位成本最低模型
Fixed Rule	任务标签到模型的固定映射
KunRoute	$Q_safe \geq \tau \rightarrow \min \text{cost} \rightarrow \min \text{latency}$
Oracle	直接使用测试集全模型真值做最优选择

10. 标准算例：完整展示 model* 如何产生

输入：解释缓存机制的基本原理，并举一个实际例子。

质量阈值： $\tau = 0.80$ 。Router 对 10 个候选一次评分，结果如下。

模型	Q_safe	预计成本	预计延迟 ms	质量达标
Model1	0.804	2.052	621	是
Model2	0.835	1.442	708	是
Model3	0.791	2.352	880	否
Model4	0.878	2.313	1012	是
Model5	0.890	3.587	1080	是
Model6	0.882	4.105	1189	是
Model7	0.902	4.996	1295	是
Model8	0.904	6.822	1343	是
Model9	0.904	9.657	1518	是
Model10	0.904	13.090	1671	是

路由结果 Model2。其 $Q_safe=0.835 \geq 0.80$ ；在质量达标候选中预计成本最低，因此直接输出该模型。

11. 工程实现与提交包结构

目录/文件	作用
docs/	Word 与 PDF 技术方案
config/model_registry.csv	Model1~Model10 单位成本、平均延迟、任务标签
config/router_config.json	质量阈值、LightMLP、校准和在线融合参数
data/sample_history.csv	5,000 条标准五元组 D
src/generate_sample_data.py	生成标准五元组数据
src/train_router.py	训练 LightMLP、聚合历史先验、生成校准表
src/router.py	在线 Q/C/L 预测与 model* 决策
src/evaluate.py	冻结测试集评估
cpp/router_kernel.cpp	AArch64 NEON + OpenMP 推理内核
tests/test_router.py	特征、批量评分、路由、在线更新单元测试
artifacts/	训练后的权重、标准化参数、质量校准表
results/	标准算例、评估结果、C++ 内核自检输出

11.1 在线 API

```
接口格式 POST /route
{
  "query": "解释缓存机制的基本原理，并举一个实际例子。",
  "quality_threshold": 0.80
}

200 OK
{
  "model": "Model2",
  "quality_safe": 0.835,
  "pred_cost": 1.442,
  "pred_latency_ms": 708,
  "reason": "QUALITY_PASS_MIN_COST"
}
```

12. 提交完整性清单

检查项	状态
M=5~10 模型，模型属性完整	Model1~Model10 共 10 个模型，单位成本/平均延迟/任务标签齐全
历史数据 D	sample_history.csv 严格包含 query/model/quality_score/latency/cost
轻量级 Router	74->48->3 LightMLP；共享模型条件评分
具体路由算法	质量可行集合 -> 最低成本 -> 最低延迟
在线性能估算	质量 EWMA + 延迟 EWMA 与离线预测融合
特征设计表	第 4 节
路由器架构图	图 1、图 3
核心算法伪代码	算法 1、算法 2
鲲鹏 NEON	cpp/router_kernel.cpp AArch64 NEON intrinsics
鲲鹏多核	OpenMP 请求级并行
可运行工程	生成数据 -> 训练 -> 评估 -> 在线路由 -> C++ 内核均提供脚本/源码

最终结论 KunRoute 把赛题转化为一个清晰的约束路由问题：历史五元组训练轻量 Q/C/L 预测器；新请求先通过质量门槛，再在合格模型中选择最低成本，成本等价时选择最低延迟；在线推理在鲲鹏 CPU 上通过 NEON 与多核并行完成。

附录 A. 标准五元组 CSV 格式

```
CSV query,model,quality_score,latency,cost
"解释缓存机制的基本原理，并举一个实际例子。",Model1,0.8621,692.40,1.7420
"解释缓存机制的基本原理，并举一个实际例子。",Model2,0.8846,735.20,1.8650
...
```

附录 B. 一键运行命令

```
运行顺序 python src/generate_sample_data.py
python src/train_router.py
python src/evaluate.py
python src/router.py
python -m unittest discover -s tests -v

cmake -S cpp -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
OMP_NUM_THREADS=8 ./build/kunroute_bench artifacts/weights.bin 100000
```